

COMP1013 Analytics Programming — Assignment Autumn 2026

Rohan Shrestha | Student ID: 22172810

29 May 2026

Contents

1	Data Loading and Preparation	3
2	Task 1 — Player Engagement Analysis by Experience Level	5
2.1	Task Explanation	5
2.2	Assumptions	5
2.3	Data Wrangling	5
2.4	Results Table	6
2.5	Visualisation	6
2.6	Interpretation	7
3	Task 2 — Game Genre Analysis	8
3.1	Task Explanation	8
3.2	Assumption	8
3.3	Data Wrangling	8
3.4	Results Table	8
3.5	Visualisation	9
3.6	Interpretation	10
4	Task 3 — Top Players and Their Behaviour	11
4.1	Task Explanation	11
4.2	Assumptions	11
4.3	Data Wrangling	11
4.4	Results Table	11
4.5	Visualisation	12
4.6	Interpretation	13

5 Task 4 — Age Group Behaviour Comparison	14
5.1 Task Explanation	14
5.2 Assumptions	14
5.3 Data Wrangling	14
5.4 Results Table	15
5.5 Visualisation	15
5.6 Interpretation	16
6 Summary	18

Declaration

By including this statement, we the authors of this work, verify that: We hold a copy of this assignment that we can produce if the original is lost or damaged. We hereby certify that no part of this assignment/product has been copied from any other student's work or from any other source except where due acknowledgement is made in the assignment. No part of this assignment/product has been written/produced for us by another person except where such collaboration has been authorised by the subject lecturer/tutor concerned. We are aware that this work may be reproduced and submitted to plagiarism detection software programs for the purpose of detecting possible plagiarism (which may retain a copy on its database for future plagiarism checking). We hereby certify that we have read and understand what the School of Computing, Engineering and Mathematics defines as minor and substantial breaches of misconduct as outlined in the learning guide for this unit.

1 Data Loading and Preparation

```
# Load all required libraries
library(tidyverse)    # data wrangling (dplyr) and visualisation (ggplot2)
library(lubridate)    # date handling - year(), as.Date()
library(knitr)        # kable() for tables
library(kableExtra)   # enhanced table formatting (striped, hover styles)
```

```
# -----
# Load the three CSV datasets
# Ensure all three files are in the same working directory as this .Rmd
# -----
players <- read.csv("players.csv", stringsAsFactors = FALSE)
games   <- read.csv("games.csv",   stringsAsFactors = FALSE)
sessions <- read.csv("sessions.csv", stringsAsFactors = FALSE)

# Convert signup_date from string to R Date type for date arithmetic
players$signup_date <- as.Date(players$signup_date)
```

```
# Quick inspection of each dataset
cat("players  :", nrow(players), "rows,", ncol(players), "columns\n")
```

```
## players : 5000 rows, 4 columns
```

```
cat("games   :", nrow(games), "rows,", ncol(games), "columns\n")
```

```
## games   : 50 rows, 4 columns
```

```
cat("sessions :", nrow(sessions), "rows,", ncol(sessions), "columns\n")
```

```
## sessions : 20000 rows, 5 columns
```

```
# Check for NA values in all three datasets
cat("\nNA counts - players:\n"); print(colSums(is.na(players)))
```

```
##
## NA counts - players:
```

```
## player_id      age      country signup_date
##           0           0           0           0
```

```
cat("\nNA counts - games:\n"); print(colSums(is.na(games)))
```

```
##
```

```
## NA counts - games:
```

```
##   game_id game_name   genre platform  
##         0         0         0         0
```

```
cat("\nNA counts - sessions:\n"); print(colSums(is.na(sessions)))
```

```
##
```

```
## NA counts - sessions:
```

```
##      session_id      player_id      game_id play_time_minutes  
##              0              0              0              0  
##           score  
##              0
```

No NA values were found in any of the three datasets, so no imputation is required. NA handling is still applied defensively throughout each task using `na.rm = TRUE` and `filter(!is.na(...))` to guard against any unexpected missing values.

2 Task 1 — Player Engagement Analysis by Experience Level

2.1 Task Explanation

This task groups players by how long they have been registered on the platform, based on the year of their `signup_date`. The three groups are:

- **Veteran**: signed up *before* 2023 — the most experienced players.
- **Intermediate**: signed up *during* 2023 — moderately experienced.
- **New**: signed up *after* 2023 — the newest players.

For each group we calculate the number of players, average session play time (minutes), and average score to understand whether experience level influences engagement behaviour.

2.2 Assumptions

- A player's experience level is determined solely by signup year, not by number of sessions.
- Players without a valid signup year are excluded.
- All sessions belonging to a player contribute equally to that player's group averages.

2.3 Data Wrangling

```
# Step 1: Classify each player into an experience group based on signup year
players_exp <- players %>%
  mutate(
    signup_year      = year(signup_date),           # extract integer year
    experience_level = case_when(
      signup_year < 2023 ~ "Veteran",              # registered before 2023
      signup_year == 2023 ~ "Intermediate",         # registered in 2023
      signup_year > 2023 ~ "New",                  # registered after 2023
      TRUE           ~ NA_character_               # catch-all for unexpected NAs
    )
  ) %>%
  filter(!is.na(experience_level))                # remove any unclassified rows

# Step 2: Join players with sessions on player_id (left join keeps all players)
player_sessions <- players_exp %>%
  left_join(sessions, by = "player_id")

# Step 3: Summarise key metrics per experience level
task1_summary <- player_sessions %>%
  group_by(experience_level) %>%
  summarise(
    num_players      = n_distinct(player_id),      # unique player count
    avg_play_time_min = round(mean(play_time_minutes, na.rm = TRUE), 2), # avg session length
```

```

    avg_score          = round(mean(score, na.rm = TRUE), 2)           # avg score per session
  ) %>%
  # Order factor levels logically: most to least experienced
  mutate(experience_level = factor(experience_level,
    levels = c("Veteran", "Intermediate", "New"))) %>%
  arrange(experience_level)

```

2.4 Results Table

```

# Display results as a formatted kable table
task1_summary %>%
  kable(
    col.names = c("Experience Level", "Number of Players",
                  "Avg Play Time (min)", "Avg Score"),
    caption   = "Player Engagement Metrics by Experience Level",
    booktabs  = TRUE
  ) %>%
  kable_styling(latex_options = c("striped", "hold_position"),
    full_width = FALSE)

```

Table 1: Player Engagement Metrics by Experience Level

Experience Level	Number of Players	Avg Play Time (min)	Avg Score
Veteran	1786	92.26	501.04
Intermediate	1851	91.73	498.20
New	1363	92.41	495.96

2.5 Visualisation

```

ggplot(task1_summary,
  aes(x = experience_level, y = avg_play_time_min,
    fill = experience_level)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(aes(label = avg_play_time_min),
    vjust = -0.4, size = 4.5, fontface = "bold") +
  scale_fill_manual(values = c("Veteran" = "#2C7BB6",
    "Intermediate" = "#ABD9E9",
    "New" = "#FDAE61")) +
  labs(
    title = "Average Play Time by Player Experience Level",
    subtitle = "Grouped by year of account signup",
    x = "Experience Level",

```

```

y      = "Average Play Time (minutes)"
) +
theme_minimal(base_size = 13) +
theme(plot.title = element_text(face = "bold"))

```

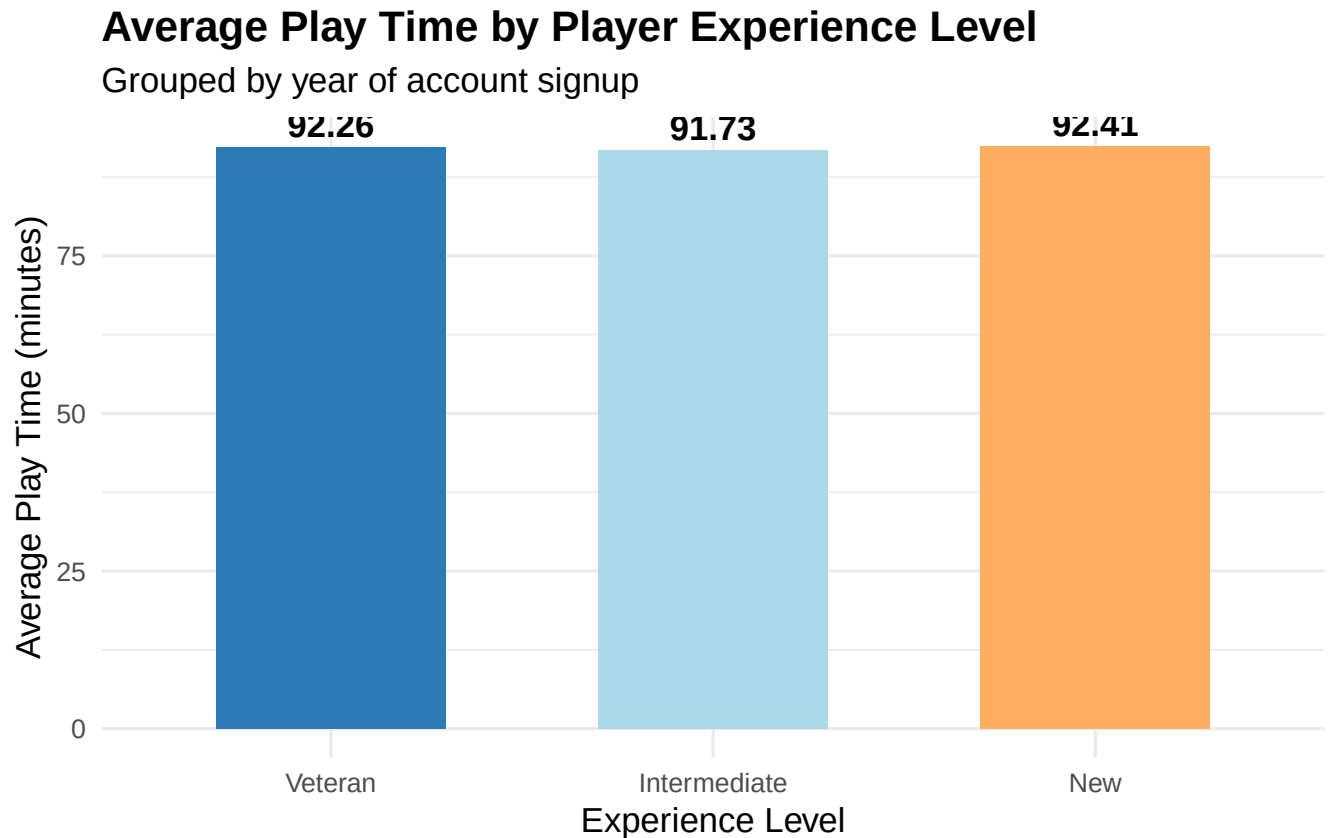


Figure 1: Average Play Time by Player Experience Level

2.6 Interpretation

The results show that both play time and average score are remarkably consistent across all three experience levels (~91-92 minutes per session; ~496-501 score). Veterans do score marginally higher than new players (501.04 versus 495.96), suggesting a slight advantage from experience - but one that is small. This indicates that **engagement on the platform is not strongly driven by seniority**: all player groups

are highly active and perform similarly. For Play Pulse Studios this is positive news - new players are not disadvantaged in terms of retention.

3 Task 2 — Game Genre Analysis

3.1 Task Explanation

This task looks at player engagement within each of the five game genres within the platform: Action, Puzzle, RPG, Sports and Strategy. For each of these genres we calculate average play time, score, number of sessions and unique players. This helps to reveal which genres are most popular and engaging.

3.2 Assumption

- Sessions are linked to genres via `game_id` → `games` table.
- Sessions where `game_id` does not match any game (and thus has no genre) are excluded.
- “Popularity” is measured by number of unique players; “engagement” also considers play time.

3.3 Data Wrangling

```
# Step 1: Join sessions with games to get genre for every session
sessions_genres <- sessions %>%
  left_join(games, by = "game_id") %>%
  filter(!is.na(genre))          # drop any sessions without a matched genre

# Step 2: Summarise engagement metrics per genre
task2_summary <- sessions_genres %>%
  group_by(genre) %>%
  summarise(
    avg_play_time      = round(mean(play_time_minutes, na.rm = TRUE), 2),
    avg_score           = round(mean(score, na.rm = TRUE), 2),
    num_sessions        = n(),                                     # total session count
    num_unique_players  = n_distinct(player_id)                   # distinct players per genre
  ) %>%
  arrange(desc(num_unique_players))                               # sort by popularity
```

3.4 Results Table

```
task2_summary %>%
  kable(
    col.names = c("Genre", "Avg Play Time (min)", "Avg Score",
                  "Sessions", "Unique Players"),
    caption   = "Player Engagement Metrics by Game Genre",
    booktabs  = TRUE
  ) %>%
  kable_styling(latex_options = c("striped", "hold_position"),
                full_width = FALSE)
```


Table 2: Player Engagement Metrics by Game Genre

Genre	Avg Play Time (min)	Avg Score	Sessions	Unique Players
Puzzle	91.67	496.52	6371	3606
Sports	92.59	503.38	3995	2769
Strategy	92.88	500.84	3643	2599
Action	92.10	493.06	3199	2370
RPG	91.41	499.69	2792	2139

3.5 Visualisation

```
ggplot(task2_summary,
       aes(x = reorder(genre, num_unique_players),
           y = num_unique_players, fill = genre)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(aes(label = num_unique_players),
            hjust = -0.2, size = 4.2) +
  coord_flip() +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Number of Unique Players by Game Genre",
    subtitle = "Based on all recorded session data",
    x = "Genre",
    y = "Number of Unique Players"
  ) +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```

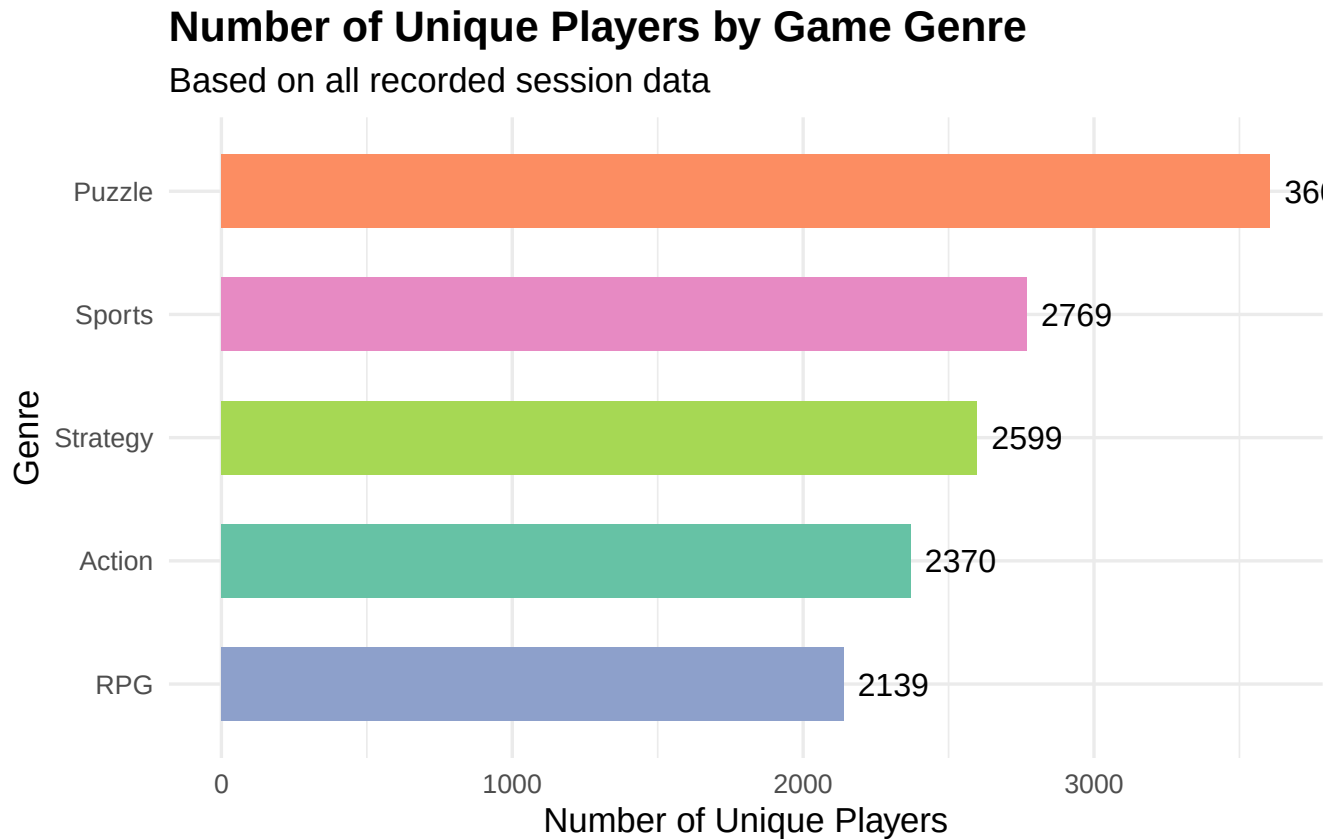


Figure 2: Number of Unique Players by Game Genre

3.6 Interpretation

Puzzle is the most popular genre with 3,606 unique players and 6,371 sessions - nearly double the session count of RPG. Sports and Strategy come in at second and third place compared to unique players. RPG fared least well in both unique players (2,139) and sessions (2,792).

Strategy shows the highest average play time - showing that strategy genre players invest more time in each session. Sports comes highest in average score though indicating skilled and competitive players.

Given this audience, the direction for Play Pulse Studios would be to invest further in Puzzle genre content given such broad appeal - while also discovering what makes Strategy so immersive for the studio to replicate within other genres with lower performance ratings like RPG.

4 Task 3 — Top Players and Their Behaviour

4.1 Task Explanation

This task identifies the 10 players with the most total play time across all sessions. Each of these players is then analysed in terms of their frequency of sessions, play time per session and score. The intention is to understand what a “power user” looks like on the platform.

4.2 Assumptions

- Total play time is the sum of all `play_time_minutes` for each player across all sessions.
- Players with any NA play time values are excluded from the total calculation.
- After identifying the top 10, all their sessions (including any with NA scores) are included in behavioural analysis, with NAs handled via `na.rm = TRUE`.

4.3 Data Wrangling

```
# Step 1: Rank all players by total play time and keep the top 10
top10_player_ids <- sessions %>%
  filter(!is.na(play_time_minutes)) %>%      # exclude any NA play times
  group_by(player_id) %>%
  summarise(total_play_time = sum(play_time_minutes)) %>%
  slice_max(total_play_time, n = 10) %>%      # top 10 by cumulative play time
  pull(player_id)                            # extract just the IDs

# Step 2: Filter sessions to top 10 players and compute behavioural metrics
task3_summary <- sessions %>%
  filter(player_id %in% top10_player_ids) %>%
  group_by(player_id) %>%
  summarise(
    total_sessions      = n(),
    avg_play_time_min   = round(mean(play_time_minutes, na.rm = TRUE), 2),
    avg_score           = round(mean(score, na.rm = TRUE), 2)
  ) %>%
  arrange(desc(avg_play_time_min))
```

4.4 Results Table

```
task3_summary %>%
  kable(
    col.names = c("Player ID", "Total Sessions",
                  "Avg Play Time (min)", "Avg Score"),
    caption   = "Behaviour Summary of Top 10 Players by Total Play Time",
    booktabs  = TRUE
```

```
) %>%
kable_styling(latex_options = c("striped", "hold_position"),
              full_width = FALSE)
```

Table 3: Behaviour Summary of Top 10 Players by Total Play Time

Player ID	Total Sessions	Avg Play Time (min)	Avg Score
3332	9	133.11	528.00
3391	9	130.11	535.11
3812	9	129.67	429.11
2809	10	119.40	418.50
4840	10	115.60	433.50
2447	13	115.54	617.69
2423	13	106.85	452.46
103	11	104.91	438.18
4175	13	99.85	504.31
2135	12	95.92	555.83

4.5 Visualisation

```
sessions %>%
  filter(player_id %in% top10_player_ids) %>%
  mutate(player_id = factor(player_id)) %>% # treat ID as a categorical variable
  ggplot(aes(x = player_id, y = score, fill = player_id)) +
    geom_boxplot(show.legend = FALSE,
                 outlier.colour = "red", outlier.shape = 16, outlier.size = 2) +
    scale_fill_brewer(palette = "Paired") +
    labs(
      title = "Score Distribution for Top 10 Players",
      subtitle = "Players ranked by cumulative play time; red dots = outliers",
      x = "Player ID",
      y = "Score"
    ) +
    theme_minimal(base_size = 13) +
    theme(plot.title = element_text(face = "bold"),
          axis.text.x = element_text(angle = 45, hjust = 1))
```

Score Distribution for Top 10 Players

Players ranked by cumulative play time; red dots = outliers

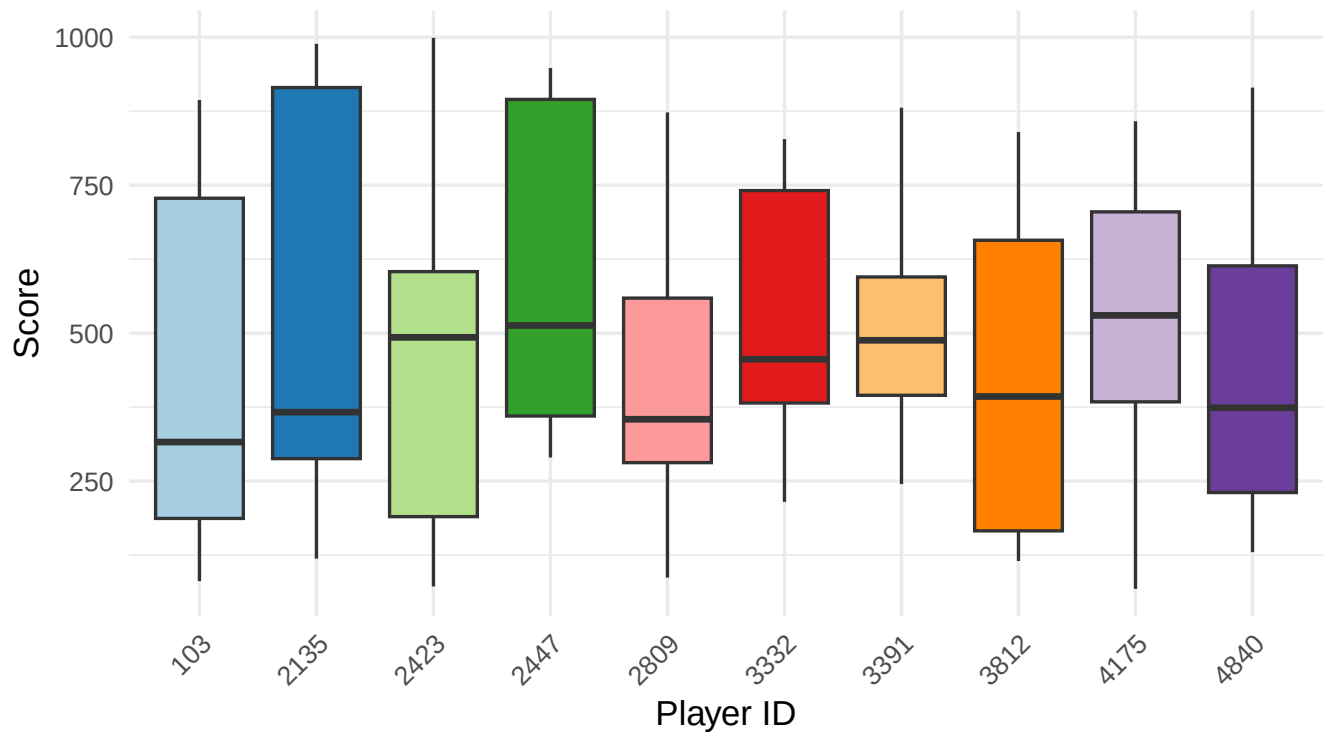


Figure 3: Score Distribution for the Top 10 Players (Boxplot)

4.6 Interpretation

The top 10 players show varied behaviour - yet all are high-engagement users. The very top player (ID 3332) sits at 133 mins per session - well above platform average of ~92 minutes. Session counts range from 9 to 13 showing a frequent return to the platform.

The boxplots reveal some notable variability within scores - players such as 2447 who score an average of 617 seem consistently high-performing compared to users such as 3812 whose average score is 429 yet who put in lengthy sessions. Such outcomes suggest that high levels of play time do not always correlate with performance - some play learning or exploring modes or play casually over longer periods.

5 Task 4 — Age Group Behaviour Comparison

5.1 Task Explanation

This task divides all players into four age brackets and compares their gaming behaviour. The goal is to determine whether age impacts either how actively or skillfully players interact with this platform. The age groups are:

- **16–25**: Youngest players
- **26–35**: Young adults
- **36–45**: Middle-aged players
- **46+**: Older players

For each group we calculate average play time, average score, and number of sessions.

5.2 Assumptions

- Each player's age is taken as their current age at the time of analysis.
- Players with missing age values are left out.
- All sessions contribute to the statistics of that age group player's statistics.

5.3 Data Wrangling

```
# Step 1: Assign each player to an age bracket
players_age <- players %>%
  filter(!is.na(age)) %>%           # exclude players with missing age
  mutate(
    age_group = case_when(
      age >= 16 & age <= 25 ~ "16-25",
      age >= 26 & age <= 35 ~ "26-35",
      age >= 36 & age <= 45 ~ "36-45",
      age >= 46             ~ "46+",
      TRUE                  ~ NA_character_ # catch unexpected values
    )
  ) %>%
  filter(!is.na(age_group))         # remove any unclassified players

# Step 2: Join age groups with sessions to attach age info to each session
sessions_age <- players_age %>%
  select(player_id, age_group) %>%
  inner_join(sessions, by = "player_id")

# Step 3: Summarise gaming behaviour per age group
task4_summary <- sessions_age %>%
```

```
group_by(age_group) %>%
summarise(
  avg_play_time_min = round(mean(play_time_minutes, na.rm = TRUE), 2),
  avg_score         = round(mean(score, na.rm = TRUE), 2),
  num_sessions      = n()
) %>%
# Set factor levels to display age groups in ascending order
mutate(age_group = factor(age_group,
  levels = c("16-25", "26-35", "36-45", "46+"))) %>%
arrange(age_group)
```

5.4 Results Table

```
task4_summary %>%
  kable(
    col.names = c("Age Group", "Avg Play Time (min)",
                  "Avg Score", "Number of Sessions"),
    caption   = "Gaming Behaviour Metrics by Age Group",
    booktabs  = TRUE
  ) %>%
  kable_styling(latex_options = c("striped", "hold_position"),
    full_width = FALSE)
```

Table 4: Gaming Behaviour Metrics by Age Group

Age Group	Avg Play Time (min)	Avg Score	Number of Sessions
16-25	91.80	496.69	4413
26-35	91.57	503.53	4538
36-45	91.58	498.02	4756
46+	93.11	496.71	6293

5.5 Visualisation

```
ggplot(task4_summary,
  aes(x = age_group, y = avg_play_time_min, fill = age_group)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(aes(label = avg_play_time_min,
    vjust = -0.4, size = 4.5, fontface = "bold")) +
  scale_fill_manual(values = c("16-25" = "#66C2A5",
    "26-35" = "#FC8D62",
    "36-45" = "#8DA0CB",
    "46+" = "#E78AC3")) +
```

```
labs(
  title = "Average Play Time by Age Group",
  subtitle = "Comparison of session duration across four age brackets",
  x = "Age Group",
  y = "Average Play Time (minutes)"
) +
theme_minimal(base_size = 13) +
theme(plot.title = element_text(face = "bold"))
```

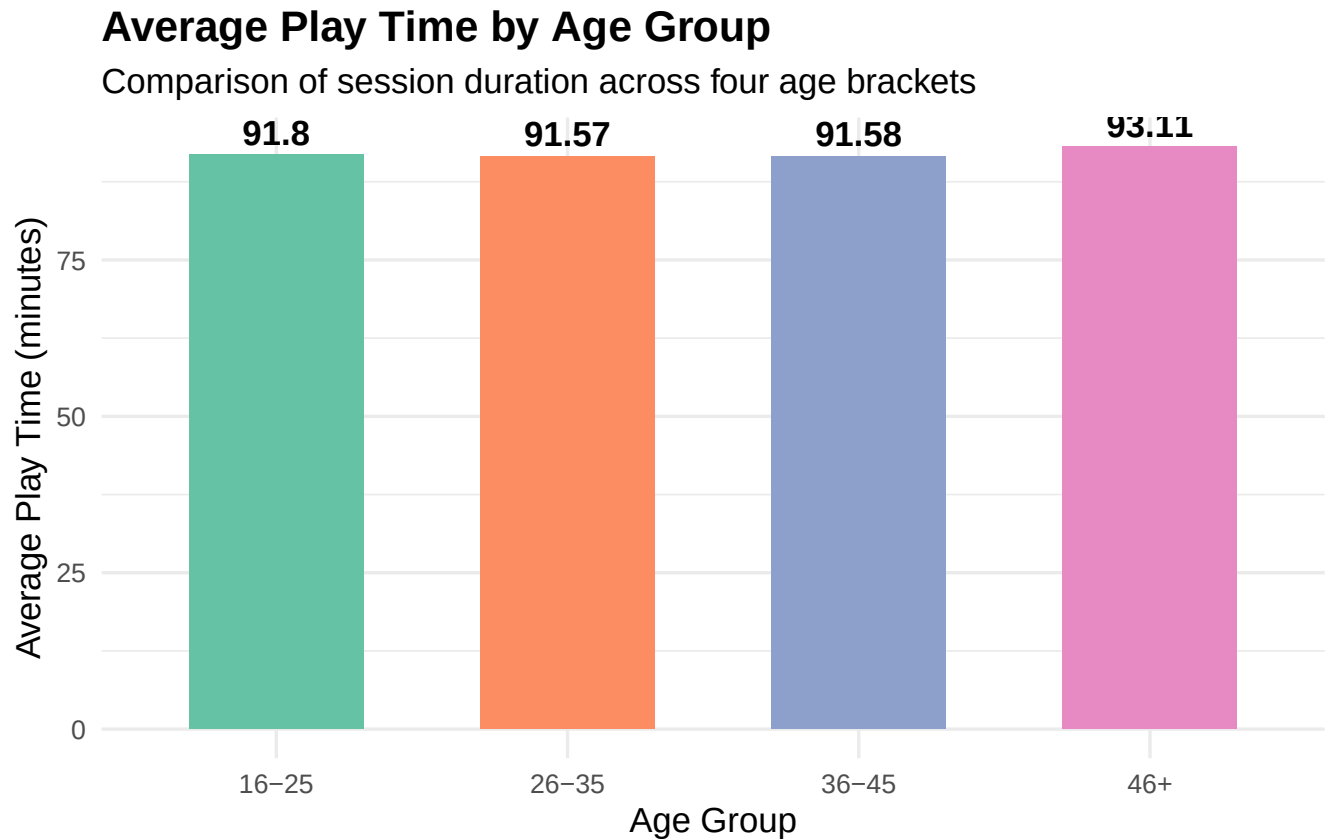


Figure 4: Average Play Time by Age Group

5.6 Interpretation

Contrary to the common assumption that younger players are more active the data tells a more subtle story. The 46+ group exhibits both the highest average play time (93.11 min) and the most sessions (6,293) - making this the most active playing cohort. The 16-25 group has the fewest sessions (4,413).

Scores are similar across age groups (~496-503) indicating no strong differences in skill levels. The 26-35 group scores a marginal highest (503.53).

While gaming activities are commonly thought of as part of a young person's activities this research

shows otherwise. For Play Pulse Studios this indicates that marketing and content development should not neglect older demographic groups - they feature a highly engaged and loyal user base.

6 Summary

Analysis of gaming data by Play Pulse Studios reveals the following insights:

1. Experience level has minimal influence upon either play time or score - all cohorts of players engage equally within the platform;
2. Puzzle is the dominant genre in terms of popularity (3,606 unique players), while Strategy genres score highest in terms of average play time;
3. Top players are highly diverse - long play times do not necessarily indicate high scoring abilities within games;
4. The most active players are of advancing ages (46+) which disproves the prevailing assumption of such engagement by younger players.